

I want to build a scalable, data-driven Selenium automation framework. Do not provide the implementation immediately. Instead, act as a Solution Architect and gather my requirements using the Socratic questioning technique. Rules: * Ask exactly one question at a time. * Each question should provide multiple-choice options (A, B, C, D...) along with a brief explanation of each option. * Wait for my response before asking the next question. * Ask only the questions necessary to fully understand my requirements. * Do not make assumptions unless I explicitly ask for recommendations. Once all the requirements are gathered: 1. Summarize my chosen requirements. 2. Explain the architecture and design decisions. 3. Provide the recommended project structure. 4. Implement the complete framework step by step with well-commented Java code. 5. Explain why each design choice was made and discuss possible alternatives. 6. Follow industry best practices for Selenium, TestNG, Maven, Page Object Model, and data-driven testing.

Great. I'll act as a Solution Architect and gather your requirements one question at a time. I won't make recommendations unless you ask for them.

Question 1: Which type of Selenium framework do you want to build?

A. Page Object Model (POM)

- Separates page locators and page actions from test logic.
- Simple, readable, and suitable for small to medium-sized projects.

B. Page Object Model + Data-Driven Framework

- Uses external data sources (Excel, JSON, CSV, etc.) for test data.
- Good for reducing duplicate test code and improving maintainability.

C. Hybrid Framework

- Combines POM, Data-Driven, Keyword-Driven, utilities, reporting, configuration management, and reusable components.
- Suitable for large enterprise applications with multiple teams.

D. Not sure (I want guidance later)

- We'll gather all requirements first.
- After that, I can recommend the most appropriate architecture based on your needs.

Please reply with one option (A, B, C, or D).

C

Noted. You selected **C. Hybrid Framework**.

A Hybrid Framework combines multiple proven design patterns and is commonly used for enterprise-scale automation because it supports modularity, reusability, maintainability, and scalability.

Question 2: Which test execution approach do you require?

A. Sequential Execution

- Tests run one after another.
- Simpler to debug.
- Suitable for smaller test suites.

B. Parallel Execution

- Multiple tests run simultaneously using TestNG.
- Reduces execution time.
- Requires thread-safe framework design.

C. Sequential + Parallel (Configurable)

- Ability to choose execution mode through TestNG XML or configuration.
- Offers maximum flexibility for different environments.

D. Distributed Execution (Selenium Grid/Cloud)

- Execute tests across multiple machines, browsers, and operating systems.
- Suitable for CI/CD pipelines and large-scale enterprise automation.

Please reply with one option (A, B, C, or D).

Noted. You selected **D. Custom Observability Platform**.

The framework should include enterprise-grade observability with:

- Structured logging
- Screenshots and browser artifacts
- Browser console and network logs (where supported)
- WebDriver event logging
- Execution metrics
- Automatic Allure attachments
- Integration with centralized observability platforms such as ELK, Splunk, or OpenTelemetry

Question 11: How extensible should the framework be for future automation types?

A. UI Automation Only

- Focus exclusively on Selenium UI testing.

B. UI + API Automation

- Combine Selenium with REST API testing in the same framework.

C. UI + API + Database

- Support UI, API, and database validation within a unified architecture.

D. Enterprise Multi-Layer Automation Platform

- A modular framework that supports UI (Selenium), API, database, messaging (e.g., Kafka/RabbitMQ), mobile automation (Appium), and can be extended to performance or contract testing without major architectural changes.

Please reply with one option (A, B, C, or D).

C

Noted. You selected **C. UI + API + Database**.

The framework should provide a unified architecture for:

- **UI automation** with Selenium
- **API testing** (e.g., REST Assured)
- **Database validation** (e.g., JDBC)
- Shared configuration, reporting, logging, and test data management across all three layers

Question 12 (Final): What Java version should the framework target?

A. Java 11

- Long-Term Support (LTS).
- Common in many enterprise environments.

B. Java 17

- Current LTS with modern language features and broad enterprise adoption.
- A strong choice for new frameworks.

C. Java 21

- Latest LTS with the newest language enhancements.

- Best for organizations already adopting the latest Java ecosystem.

D. I have a specific version in mind

- Specify the exact Java version required by your organization or project.

Please reply with one option (A, B, C, or D).

C

Noted. You selected **C. Java 21**.

This completes the requirements gathering.

Summary of Your Selected Requirements

Category	Your Choice
Framework Type	Hybrid Framework
Execution	Distributed Execution (Grid/Cloud)
Test Data	Multiple Configurable Sources (Excel, JSON, CSV, DB, etc.)
Reporting	Allure Reports
Dependency Injection	Spring Framework
Design Pattern	Screenplay Pattern
Browser Support	Cross-Browser + Cross-Platform
CI/CD	Enterprise CI/CD
Configuration	TestNG XML + Properties/YAML + Maven Parameters
Observability	Enterprise + ELK/Splunk/OpenTelemetry
Automation Scope	UI + API + Database
Java Version	Java 21

In the next step, I'll use these requirements to design an **enterprise-grade, scalable automation platform** and then implement it incrementally with:

1. Architecture overview and rationale.
2. High-level component diagram.
3. Maven multi-module project structure.
4. Design decisions and alternatives.
5. Spring Boot integration.
6. Selenium + TestNG + Screenplay implementation.
7. Distributed execution using Selenium Grid.
8. Cross-browser support.
9. Data-driven layer (Excel/JSON/CSV/DB).
10. REST Assured integration.

11. Database utilities.
12. Allure reporting.
13. Logging and observability.
14. CI/CD pipeline examples.
15. Complete, production-ready Java 21 code with detailed explanations of every class and design choice.